

Práctica 2B

Datos de casas

Aprendizaje No Supervisado y Visualización

Unidad 2 · Maestría en Ciencia de Datos e Información

Bryan Rodríguez Abarca

Matrícula 26DF1185A9

1 de octubre de 2026

INFOTEC · Ciudad de México

Un bloque censal con 35682 habitantes fue lo primero que me llamó la atención de California Housing, y al lado una columna de cuartos por vivienda que se mueve entre 0.33 y 34. Las ocho columnas del archivo están en unidades que no se hablan entre sí, y eso condiciona los cuatro aplanados a dos dimensiones que hay que hacer con ellas —PCA, UMAP, t-SNE y un autoencoder— más que la elección del método.

El precio no participa en ninguno de los cuatro. Ni el escalado ni PCA ni UMAP ni t-SNE ni la red reciben `y`; entra al final, como color.

```
import sys
import numpy as np
import sklearn
import umap
import tensorflow

print("python", sys.version.split()[0])
print("numpy", np.__version__)
print("scikit-learn", sklearn.__version__)
print("umap-learn", umap.__version__)
print("tensorflow", tensorflow.__version__)
```

```
python 3.11.15
numpy 2.4.6
scikit-learn 1.9.0
umap-learn 0.5.12
tensorflow 2.21.0
```

1. Los datos

El código de lectura es el que viene en la actividad.

```
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
import umap

california_housing = fetch_california_housing()
X = california_housing.data
y = california_housing.target
X.shape, y.shape
```

```
((20640, 8), (20640,))
```

```
print(california_housing.feature_names)
```

```
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
 ↪  'Latitude', 'Longitude']
```

```
print("columna     mínimo mediana máximo desv.est.")
for i, c in enumerate(california_housing.feature_names):
    col = X[:, i]
    print(f"{c:11s}", round(col.min(), 2), round(np.median(col), 2),
          round(col.max(), 2), round(col.std(), 2))

print()
print("precio: mínimo", y.min(), "mediana", np.median(y),
      "máximo", y.max())
```

```
columna     mínimo mediana máximo desv.est.
MedInc      0.5 3.53 15.0 1.9
HouseAge    1.0 29.0 52.0 12.59
AveRooms    0.85 5.23 141.91 2.47
AveBedrms   0.33 1.05 34.07 0.47
Population  3.0 1166.0 35682.0 1132.43
AveOccup    0.69 2.82 1243.33 10.39
Latitude    32.54 34.26 41.95 2.14
Longitude   -124.35 -118.49 -114.31 2.0

precio: mínimo 0.14999 mediana 1.797 máximo 5.00001
```

Population va de 3 a 35682 personas y AveBedrms de 0.33 a 34 cuartos; sus desviaciones son 1132.43 y 0.47, un factor de 2400 entre una y otra. Latitude y Longitude vienen en grados, dentro de una franja de diez. Todo lo que voy a hacer aquí abajo mide distancias entre renglones, y una distancia que suma diferencias al cuadrado sobre columnas así se convierte en la diferencia de Population con algo de ruido encima.

2. El mapa que sale de las columnas tal como vienen

Antes de decidir nada, PCA sobre `X` sin tocarlo.

```
from sklearn.decomposition import PCA

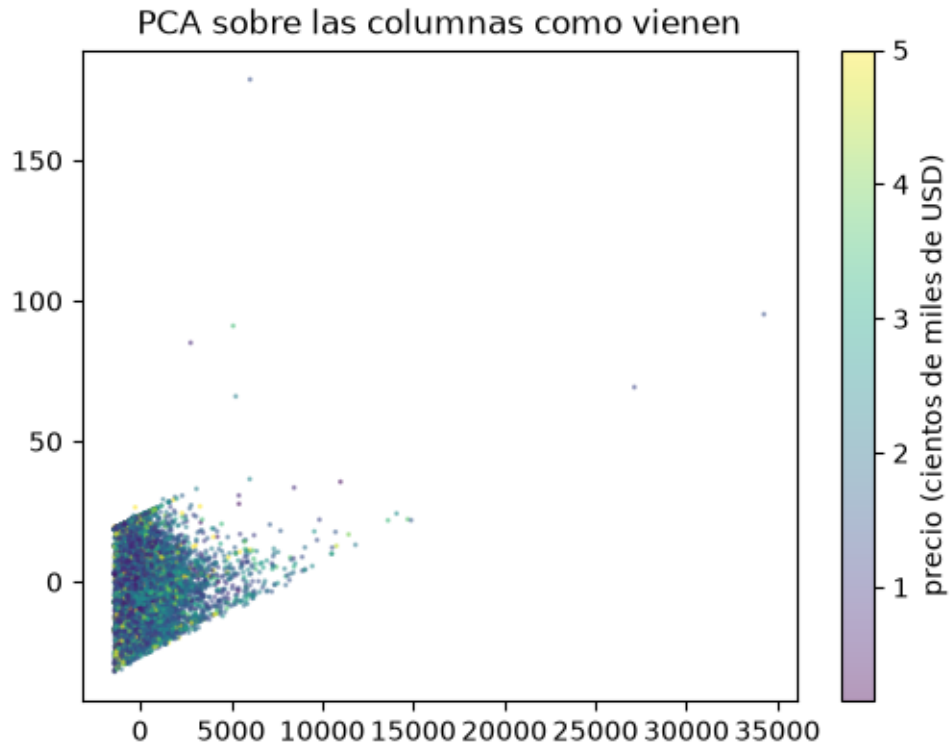
pca_crudo = PCA(n_components=2)
Z_crudo = pca_crudo.fit_transform(X)

print("varianza explicada:", pca_crudo.explained_variance_ratio_)
print("correlación del eje 1 con Population:",
      round(np.corrcoef(Z_crudo[:, 0], X[:, 4])[0, 1], 6))
mayor = np.abs(pca_crudo.components_[1]).argmax()
print("columna que domina el eje 2:",
      california_housing.feature_names[mayor],
      round(pca_crudo.components_[1][mayor], 4))
```

```
varianza explicada: [9.99789327e-01 1.13281110e-04]
correlación del eje 1 con Population: 1.0
columna que domina el eje 2: HouseAge 0.9922
```

El 99.98 % de la varianza se va en una sola componente, que correlaciona con Population en 1.0 redondeando a seis decimales. La segunda es HouseAge con carga 0.9922. Las dos componentes son dos columnas del archivo con otro nombre; el ingreso, los cuartos y las coordenadas quedaron pesando menos que el redondeo.

```
fig, ax = plt.subplots(figsize=(6.0, 4.4))
p = ax.scatter(Z_crudo[:, 0], Z_crudo[:, 1], c=y, s=1, alpha=0.4,
               cmap="viridis")
ax.set_title("PCA sobre las columnas como vienen")
fig.colorbar(p, label="precio (cientos de miles de USD)")
plt.show()
```



La nube es una cuña apretada contra el borde izquierdo, estirada hasta 35000 por el puñado de bloques con más población, y los colores están revueltos por todos lados.

3. Una medida del color

Decir “no se ve nada” no me sirve para comparar cuatro mapas entre sí. Lo que le pido a un mapa pintado con el precio es que quien caiga cerca de mí cueste parecido a mí, así que mido eso: para cada renglón busco sus 20 vecinos más cercanos en el mapa y promedio la diferencia absoluta de precio contra ellos. La medida es mía, no de la actividad, y es local: solo mira el vecindario de cada punto. El 20 lo elegí sin más criterio que el orden de magnitud, y más abajo repito la comparación con 100.

```
from sklearn.neighbors import NearestNeighbors


def vecinos(Z, k=20):
    nn = NearestNeighbors(n_neighbors=k + 1).fit(Z)
    _, idx = nn.kneighbors(Z)
    return idx[:, 1:]  # la columna 0 es el propio punto


def precio_vecino(Z, k=20):
    v = vecinos(Z, k)
    return np.abs(y[v] - y[:, None]).mean()


ordenado = np.sort(y)
i = np.arange(len(y))
pares = len(y) * (len(y) - 1) / 2
print("todos los pares del conjunto:",
      round((ordenado * (2 * i - len(y) + 1)).sum() / pares, 4))
print("vecinos en las 8 columnas crudas:", round(precio_vecino(X), 4))
```

```
todos los pares del conjunto: 1.2593
vecinos en las 8 columnas crudas: 1.1281
```

Dos renglones cualesquiera del conjunto se llevan 1.2593 de precio. Los 20 vecinos más cercanos en las ocho columnas crudas se llevan 1.1281, un 10 % menos.

4. Escalar

Escalo. Con Population entre 3 y 35682, la distancia era Population. Escalar tiene su precio: después de dividir cada columna entre su desviación, un grado de latitud y una persona por hogar valen lo mismo dentro de la distancia.

```
from sklearn.preprocessing import StandardScaler

Xs = StandardScaler().fit_transform(X)

print("desviaciones después de escalar:", np.round(Xs.std(axis=0), 3))
print("vecinos en las 8 columnas escaladas:", round(precio_vecino(Xs), 4))
```

```
desviaciones después de escalar: [1. 1. 1. 1. 1. 1. 1. 1.]
vecinos en las 8 columnas escaladas: 0.6078
```

La misma cuenta sobre las columnas escaladas da 0.6078 contra los 1.1281 de antes: la diferencia de precio entre vecinos cae casi a la mitad sin que haya entrado ningún dato nuevo. Con las columnas crudas ninguna proyección tenía de dónde sacar zonas caras y baratas.

Ese 0.6078 es lo que sabe del precio la distancia con las ocho columnas parejas, y lo llevo de referencia para los cuatro mapas. De aquí en adelante todo corre sobre `Xs`.

5. PCA

```
pca = PCA(n_components=2)
Z_pca = pca.fit_transform(Xs)

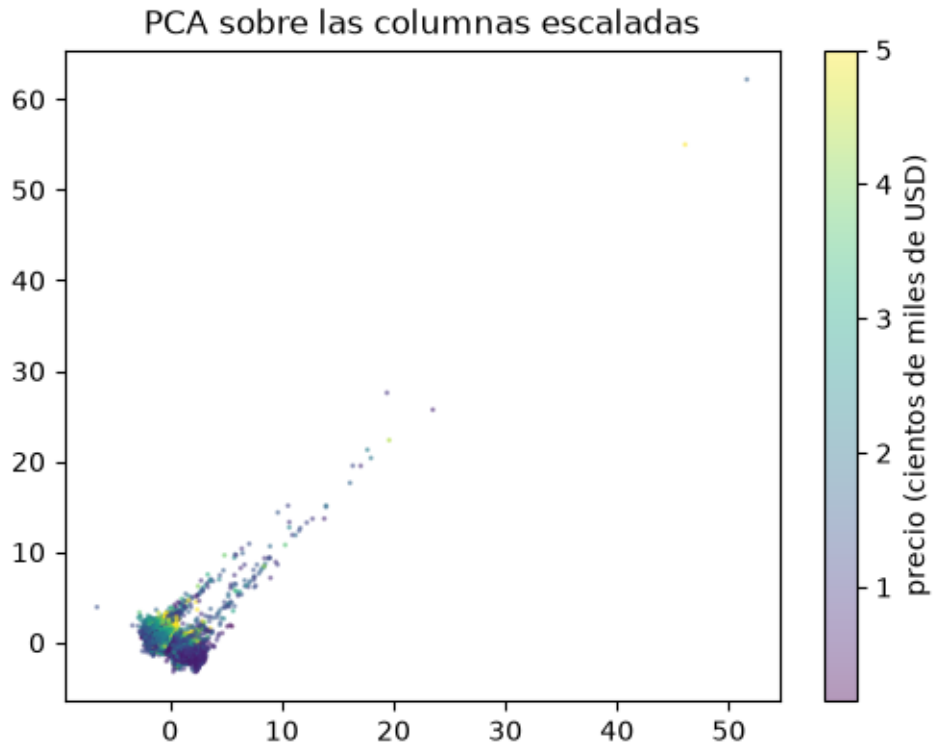
print("varianza explicada:", np.round(pca.explained_variance_ratio_, 4),
      "suma", round(pca.explained_variance_ratio_.sum(), 4))
print()
print("columna     eje 1  eje 2")
for i, c in enumerate(california_housing.feature_names):
    print(f"{c:11s}", round(pca.components_[0][i], 3),
          round(pca.components_[1][i], 3))
```

```
varianza explicada: [0.2534 0.2352] suma 0.4885

columna     eje 1  eje 2
MedInc      0.077 0.23
HouseAge    0.007 -0.261
AveRooms    0.439 0.547
AveBedrms   0.4 0.502
Population  -0.172 0.101
AveOccup    -0.015 0.003
Latitude    0.568 -0.374
Longitude   -0.537 0.422
```

Con las columnas parejas, el primer eje deja de ser una columna sola: Latitude 0.568, Longitude -0.537, AveRooms 0.439 y AveBedrms 0.400. Las dos componentes juntan el 48.85 % de la varianza.

```
fig, ax = plt.subplots(figsize=(6.0, 4.4))
p = ax.scatter(Z_pca[:, 0], Z_pca[:, 1], c=y, s=1, alpha=0.4,
               cmap="viridis")
ax.set_title("PCA sobre las columnas escaladas")
fig.colorbar(p, label="precio (cientos de miles de USD)")
plt.show()
```




```
print("eje 1: mínimo", round(Z_pca[:, 0].min(), 2),
      "máximo", round(Z_pca[:, 0].max(), 2))
print("eje 1: percentiles 0.5 y 99.5",
      np.round(np.percentile(Z_pca[:, 0], [0.5, 99.5]), 2))
print("AveOccup llega a", X[:, 5].max(), "y AveRooms a", X[:, 2].max())
```

```
eje 1: mínimo -6.55 máximo 51.61
eje 1: percentiles 0.5 y 99.5 [-2.1   4.12]
AveOccup llega a 1243.3333333333333 y AveRooms a 141.9090909090909
```

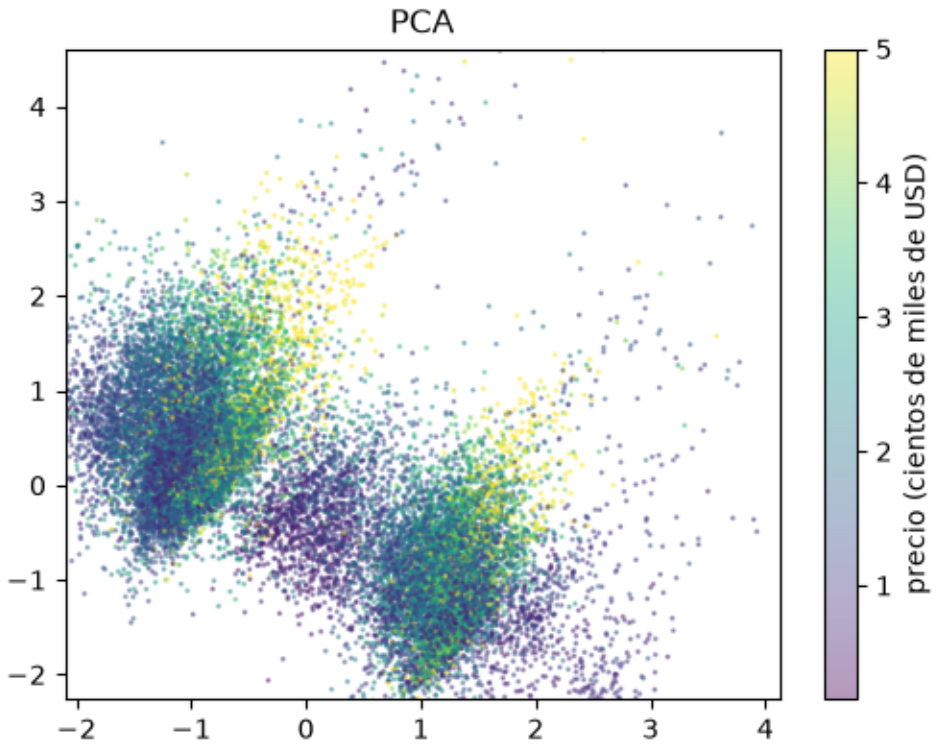
La gráfica salió peor que la anterior: una mancha negra en el origen con una cola de puntos sueltos que llega a 51.61, cuando el 99.5 % de los renglones no pasa de 4.12. Los culpables siguen en el archivo —AveOccup llega a 1243 personas por hogar y AveRooms a 141.9 cuartos—, y escalar no los borra, solo los deja a veinte o treinta desviaciones del resto.

Recorto la vista al percentil 0.5 y 99.5 de cada eje, e imprimo cuántos renglones quedan fuera del recuadro. No sale del cálculo ningún renglón: los métodos y las mediciones siguen viendo los 20640, lo único que cambia es hasta dónde llegan los ejes de la figura. El mismo recorte va para los cuatro mapas.

```
def mapa(Z, titulo):
    lo = np.percentile(Z, 0.5, axis=0)
    hi = np.percentile(Z, 99.5, axis=0)
    fuera = ((Z < lo) | (Z > hi)).any(axis=1).sum()
    fig, ax = plt.subplots(figsize=(6.0, 4.4))
    p = ax.scatter(Z[:, 0], Z[:, 1], c=y, s=1, alpha=0.4, cmap="viridis")
```

```
    ax.set_xlim(lo[0], hi[0])
    ax.set_ylim(lo[1], hi[1])
    ax.set_title(titulo)
    fig.colorbar(p, label="precio (cientos de miles de USD)")
    plt.show()
    print("renglones fuera del recuadro:", fuera)


mapa(Z_pca, "PCA")
```




```
renglones fuera del recuadro: 339
```

```
print("columna     horizontal vertical")
for j, c in [(6, "Latitude"), (7, "Longitude"), (0, "MedInc"),
             (2, "AveRooms")]:
    print(f"{c:11s}", round(np.corrcoef(Z_pca[:, 0], X[:, j])[0, 1], 3),
          round(np.corrcoef(Z_pca[:, 1], X[:, j])[0, 1], 3))

izquierda = Z_pca[:, 0] < 0
print()
print("lóbulo izquierdo:", izquierda.sum(), "renglones, latitud media",
      round(X[izquierda, 6].mean(), 2))
print("lóbulo derecho:", (~izquierda).sum(), "renglones, latitud media",
      round(X[~izquierda, 6].mean(), 2))
banda = Z_pca[:, 1] > 1
```

```
print("por encima de la altura 1:", banda.sum(), "renglones, precio medio",
      round(y[banda].mean(), 3), "contra", round(y[~banda].mean(), 3))
```

```
columna     horizontal vertical
Latitude    0.809 -0.513
Longitude   -0.764 0.579
MedInc      0.11 0.315
AveRooms    0.625 0.75

lóbulo izquierdo: 11560 renglones, latitud media 33.93
lóbulo derecho: 9080 renglones, latitud media 37.79
por encima de la altura 1: 3077 renglones, precio medio 2.554 contra 1.983
```

Recortado, el mapa tiene dos lóbulos densos, uno arriba a la izquierda y otro abajo al centro, y una zona más suelta que sube hacia la derecha. Los dos lóbulos son las dos mitades del estado: el eje horizontal correlaciona 0.809 con Latitude y −0.764 con Longitude, y los 11560 renglones a la izquierda del cero promedian latitud 33.93 —el sur, alrededor de Los Ángeles— contra 37.79 de los 9080 de la derecha, que es la bahía de San Francisco y el norte.

El precio no arma lóbulo propio. Los amarillos se apilan en la franja de arriba, la dirección de AveRooms (0.750 con el eje vertical): los 3077 renglones por encima de la altura 1 promedian 2.554 contra 1.983 del resto.

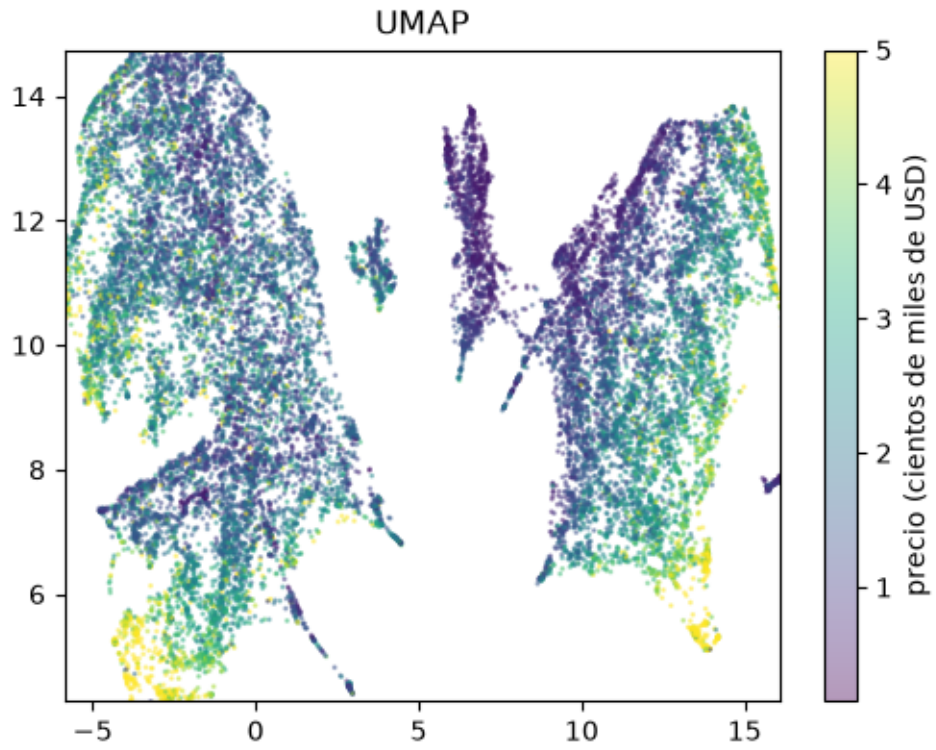
6. UMAP

```
import time

t0 = time.time()
Z_umap = umap.UMAP(n_components=2, random_state=0,
                   n_jobs=1).fit_transform(Xs)
print("UMAP tardó", round(time.time() - t0), "segundos")

mapa(Z_umap, "UMAP")
```

```
UMAP tardó 18 segundos
```




```
renglones fuera del recuadro: 416
```

UMAP parte los datos en dos bloques grandes separados por un canal vertical, con filamentos que salen hacia abajo y unas islas chicas sueltas entre los dos. El amarillo tampoco está repartido, pero se junta distinto que en PCA: ocupa el borde inferior de los dos bloques, en las puntas de los filamentos, y arriba no hay casi nada caro.

7. t-SNE

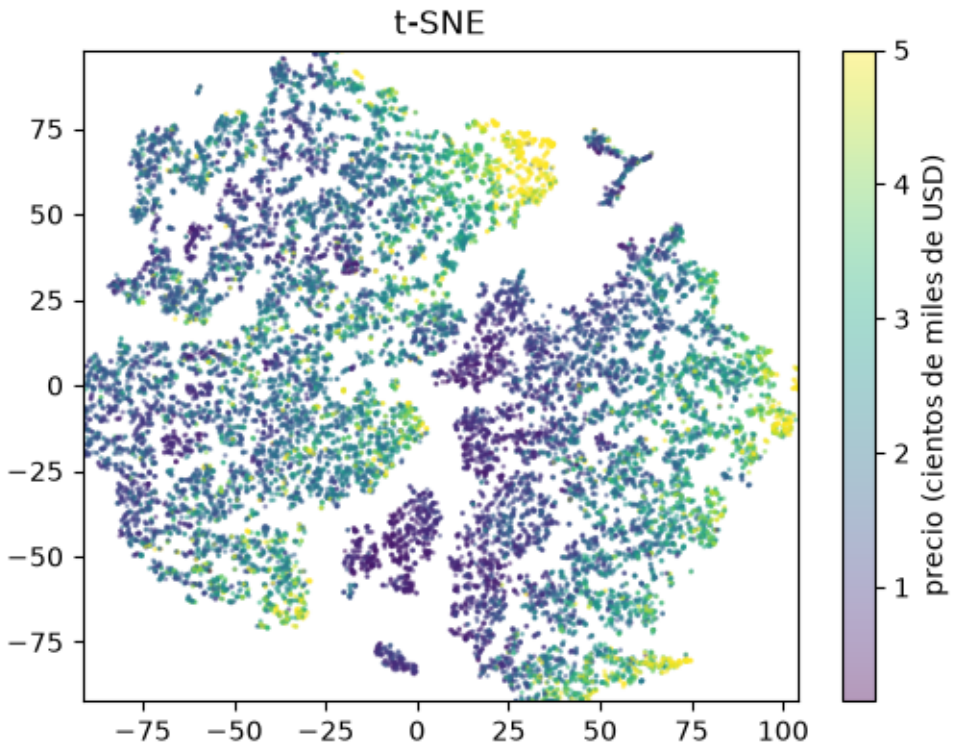
t-SNE es el más caro de los cuatro con los 20640 renglones. Una corrida con `random_state=0` y el resto por omisión.

```
from sklearn.manifold import TSNE

t0 = time.time()
Z_tsne = TSNE(n_components=2, random_state=0).fit_transform(Xs)
print("t-SNE tardó", round(time.time() - t0), "segundos con los",
      len(Xs), "renglones")

mapa(Z_tsne, "t-SNE")
```

```
t-SNE tardó 43 segundos con los 20640 renglones
```




```
renglones fuera del recuadro: 416
```

Lo que devuelve no se parece a los otros dos: un disco lleno de islas chicas, decenas, sin dos bloques ni una dirección dominante. El amarillo aparece en manchas compactas —arriba al centro, en el borde derecho, abajo— en vez de formar un gradiente.

8. Autoencoder

El autoencoder que armé es 8-4-2-4-8: de las ocho columnas a cuatro, a dos, y de vuelta por el mismo camino; los dos números del cuello son la proyección. Lo que minimiza es el error cuadrático medio contra su propia entrada, y como las columnas están escaladas ese error se puede poner al lado del que comete PCA al reconstruir desde su plano. Entreno con `adam` y lotes de 256, sobre una rejilla de dos activaciones por dos números de épocas.

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
from tensorflow import keras


def autoencoder(activacion, epocas, semilla=0):
    keras.utils.set_random_seed(semilla)
    codificador = keras.Sequential([
        keras.layers.Input(shape=(8,)),
        keras.layers.Dense(4, activation=activacion),
        keras.layers.Dense(2, activation="linear"),
```

```
    ])
    decodificador = keras.Sequential([
        keras.layers.Input(shape=(2,)),
        keras.layers.Dense(4, activation=activacion),
        keras.layers.Dense(8, activation="linear"),
    ])
    ae = keras.Sequential([codificador, decodificador])
    ae.compile(optimizer="adam", loss="mse")
    h = ae.fit(Xs, Xs, epochs=epocas, batch_size=256, verbose=0)
    return codificador.predict(Xs, verbose=0), h.history["loss"][-1]


reconstruccion = pca.inverse_transform(Z_pca)
print("MSE de reconstrucción del plano de PCA:",
      round(((Xs - reconstruccion) ** 2).mean(), 4))
print()
latentes = {}
for activacion in ["relu", "tanh"]:
    for epocas in [30, 100]:
        Z, perdida = autoencoder(activacion, epocas)
        latentes[(activacion, epocas)] = Z
        print(activacion, epocas, "épocas: pérdida", round(perdida, 4),
              "precio vecino", round(precio_vecino(Z), 4))
```

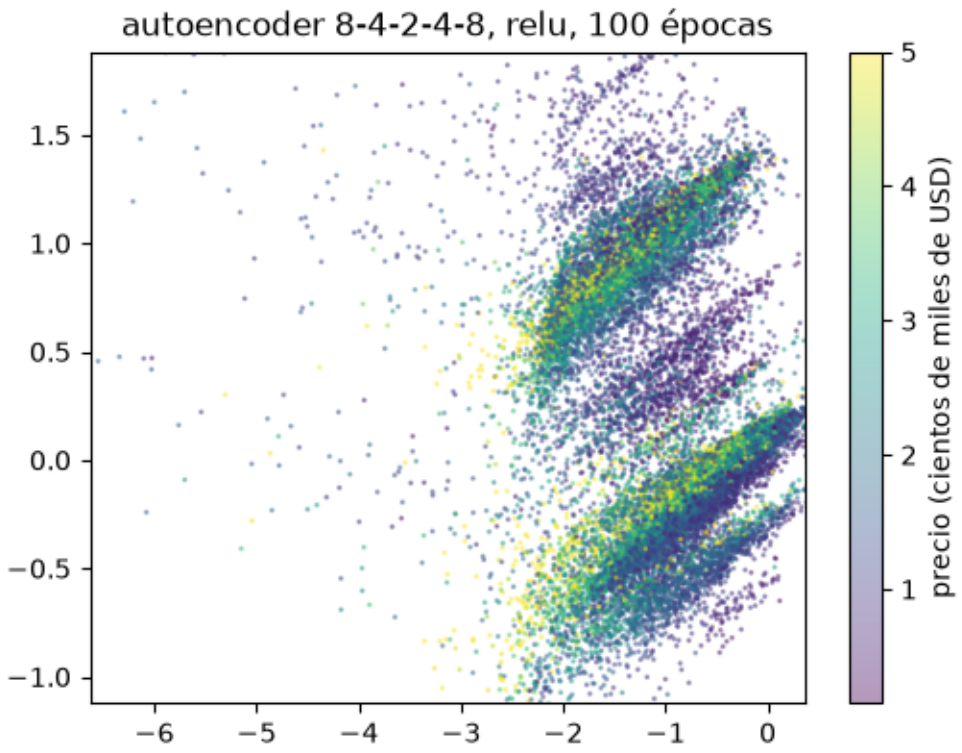
```
MSE de reconstrucción del plano de PCA: 0.5115

relu 30 épocas: pérdida 0.5457 precio vecino 1.0132
relu 100 épocas: pérdida 0.3911 precio vecino 0.972
tanh 30 épocas: pérdida 0.5758 precio vecino 0.9599
tanh 100 épocas: pérdida 0.4746 precio vecino 0.9998
```

PCA reconstruye las ocho columnas desde su plano con 0.5115 de error. Dos corridas de la red quedan por encima de ese número: `relu` con 30 épocas en 0.5457 y `tanh` con 30 en 0.5758. `tanh` con 100 baja a 0.4746 y ya le gana a PCA; `relu` con 100 llega a 0.3911 y le gana por bastante más. Me quedo con esa última, porque el error de reconstrucción es lo único que la red está optimizando.

La columna de la derecha va por su lado: las cuatro corridas dan entre 0.9599 y 1.0132 de diferencia de precio entre vecinos, y la que mejor reconstruye no es la que mejor ordena el precio.

```
Z_auto = latentes[("relu", 100)]
mapa(Z_auto, "autoencoder 8-4-2-4-8, relu, 100 épocas")
```




```
renglones fuera del recuadro: 354
```

```python
from sklearn.linear_model import LinearRegression

print("columna     horizontal vertical")
for j, c in enumerate(california_housing.feature_names):
    print(f"{c:11s}", round(np.corrcoef(Z_auto[:, 0], X[:, j])[0, 1], 3),
          round(np.corrcoef(Z_auto[:, 1], X[:, j])[0, 1], 3))

print()
print("correlación del precio con MedInc:",
      round(np.corrcoef(y, X[:, 0])[0, 1], 3))
print("R² del latente predicho desde el plano de PCA:",
      round(LinearRegression().fit(Z_pca, Z_auto).score(Z_pca, Z_auto), 3))
print("R² del plano de PCA predicho desde el latente:",
      round(LinearRegression().fit(Z_auto, Z_pca).score(Z_auto, Z_pca), 3))
```

```
columna     horizontal vertical
MedInc      -0.236 -0.177
HouseAge    0.401 0.312
AveRooms    -0.897 0.249
AveBedrms   -0.862 0.308
Population  -0.116 -0.367
AveOccup    -0.016 -0.023
Latitude    -0.245 0.879
Longitude   0.166 -0.867
```

```
correlación del precio con MedInc: 0.688
R² del latente predicho desde el plano de PCA: 0.929
R² del plano de PCA predicho desde el latente: 0.916
```

El espacio latente sale en abanico: unas bandas diagonales densas a la derecha y una cola larga hacia la izquierda, con los colores mezclados dentro de cada banda. El eje horizontal es el tamaño de la vivienda, −0.897 con AveRooms y −0.862 con AveBedrms; el vertical es la geografía, 0.879 con Latitude y −0.867 con Longitude. A MedInc, la columna que más correlaciona con el precio (0.688), le quedan −0.236 y −0.177.

Yo esperaba que la red, por no estar obligada a ser lineal, se pareciera a UMAP más que a PCA. Lo que encontró es casi el mismo plano: el latente se predice desde las dos componentes de PCA con R^2 0.929, y el plano de PCA desde el latente con 0.916. Los ejes no son los mismos —la red puso el tamaño de la vivienda en uno y la geografía en el otro, donde PCA mezcla ambas cosas en sus dos componentes—, pero el plano que las dos eligen dentro de las ocho dimensiones es casi el mismo.

```python
for semilla in [1, 2]:
    Z, perdida = autoencoder("relu", 100, semilla)
    latentes[("relu", 100, semilla)] = Z
    print("semilla", semilla, "pérdida", round(perdida, 4),
          "precio vecino", round(precio_vecino(Z), 4))

tres = [precio_vecino(latentes[("relu", 100)]),
        precio_vecino(latentes[("relu", 100, 1)]),
        precio_vecino(latentes[("relu", 100, 2)])]
print("desviación de las tres semillas:", round(np.std(tres, ddof=1), 4))
```

```
semilla 1 pérdida 0.3864 precio vecino 1.0032
semilla 2 pérdida 0.2685 precio vecino 0.9812
desviación de las tres semillas: 0.016
```

Cambiar la semilla mueve el entrenamiento —la pérdida va de 0.3911 a 0.3864 y a 0.2685— y mucho menos la lectura del precio, que se queda entre 0.9720 y 1.0032. Mi propio criterio de quedarme con la que mejor reconstruye pediría la semilla 2, que baja el error a dos tercios; sigo con la 0, que es la corrida de la rejilla, y anoto aquí que con la 2 las cifras de más abajo serían 0.9812 en la diferencia de precio y 14.8 % en la cuenta del máximo.

9. Los cuatro mapas juntos

```python
proyecciones = [("PCA", Z_pca), ("UMAP", Z_umap), ("t-SNE", Z_tsne),
                ("autoencoder", Z_auto)]

for nombre, Z in proyecciones:
    print(nombre, round(precio_vecino(Z), 4))
print()
print("8 columnas escaladas", round(precio_vecino(Xs), 4))
```

```
PCA 0.9904
UMAP 0.7125
t-SNE 0.6612
autoencoder 0.972

8 columnas escaladas 0.6078
```

t-SNE deja a cada renglón rodeado de precios que se le parecen en 0.6612, a 0.05 de las ocho columnas escaladas. UMAP queda en 0.7125. PCA (0.9904) y el autoencoder (0.9720) están más cerca de los 1.2593 de dos renglones cualesquiera que del 0.6078 de partida, y entre ellos la diferencia es 0.018, por debajo de la desviación de 0.016 que la red muestra al cambiarle la semilla: por esta medida ese par no queda ordenado.

```
print("solo Latitude y Longitude escaladas:",
      round(precio_vecino(Xs[:, 6:8]), 4))
print("solo MedInc escalada:", round(precio_vecino(Xs[:, :1]), 4))
```

```
solo Latitude y Longitude escaladas: 0.4876
solo MedInc escalada: 0.8374
```

Antes de leer el 0.6078 como una meta probé qué pasa con menos columnas. La distancia sobre Latitude y Longitude solas —el mapa de California pelón— da 0.4876, mejor que los cuatro aplanados y que las ocho columnas juntas. MedInc sola da 0.8374, mejor que PCA y que el autoencoder.

El 0.6078 entonces no es techo ni cota: es lo que dan las ocho columnas con el peso que les tocó, y tirar las que para el precio son ruido lo mejora. Lo que compara la tabla de arriba es cuánto de lo que las ocho columnas saben del precio sobrevive al aplanado.

```
for nombre, Z in proyecciones:
    print(nombre, round(precio_vecino(Z, 100), 4))
```

```
PCA 1.0025
UMAP 0.746
t-SNE 0.7285
autoencoder 0.9996
```

Con 100 vecinos en vez de 20 todos los números suben —t-SNE 0.7285, UMAP 0.7460, autoencoder 0.9996 y PCA 1.0025— y el orden se mantiene.

```
rng = np.random.default_rng(0)
y_barajado = y.copy()
rng.shuffle(y_barajado)

v = vecinos(Z_tsne)
print("t-SNE con los precios barajados:",
      round(np.abs(y_barajado[v] - y_barajado[:, None]).mean(), 4))
```

```
t-SNE con los precios barajados: 1.2585
```

```
t0 = time.time()
Z_tsne_1 = TSNE(n_components=2, random_state=1).fit_transform(Xs)
print("t-SNE con random_state=1 da el mismo arreglo:",
      np.array_equal(Z_tsne_1, Z_tsne), "en", round(time.time() - t0), "s")

Z_tsne_50 = TSNE(n_components=2, random_state=0,
                 perplexity=50).fit_transform(Xs)
print("t-SNE con perplexity=50:", round(precio_vecino(Z_tsne_50), 4))

Z_umap_50 = umap.UMAP(n_components=2, random_state=0, n_jobs=1,
                      n_neighbors=50).fit_transform(Xs)
print("UMAP con n_neighbors=50:", round(precio_vecino(Z_umap_50), 4))
```

```
t-SNE con random_state=1 da el mismo arreglo: True en 43 s
t-SNE con perplexity=50: 0.6628
UMAP con n_neighbors=50: 0.7304
```

Barajar los precios sobre el mapa de t-SNE deja la medida en 1.2585, el valor de dos renglones cualesquiera: el 0.6612 sale de cómo las ocho columnas se relacionan con el precio y no de cómo está repartido el precio.

La semilla, en cambio, no entra en la cuenta de t-SNE: la inicialización por omisión de esta versión es `init="pca"`, determinista, y correr con `random_state=1` devuelve el mismo arreglo dato por dato. Lo que mueve las cifras de estos dos métodos es el parámetro de vecindario, que les había dejado por omisión mientras a la red le pasaba una rejilla entera. t-SNE con `perplexity=50` da 0.6628, dieciséis diezmilésimas por encima de su corrida por omisión, y UMAP con `n_neighbors=50` sube a 0.7304. Mirar más lejos no mejora a ninguno de los dos ni cambia el orden entre ellos.

```
tope = y >= 5.00001
print("renglones en el máximo de 5.00001:", tope.sum())
print()

for nombre, Z in proyecciones:
    v = vecinos(Z)
    print(nombre, round(100 * tope[v][tope].mean(), 1), "%")
print()
v = vecinos(Xs)
print("8 columnas escaladas", round(100 * tope[v][tope].mean(), 1), "%")
print("proporción en el conjunto entero", round(100 * tope.mean(), 1), "%")
for semilla in [1, 2]:
    v = vecinos(latentes[("relu", 100, semilla)])
    print("autoencoder con semilla", semilla,
          round(100 * tope[v][tope].mean(), 1), "%")
```

```
renglones en el máximo de 5.00001: 965

PCA 23.7 %
UMAP 42.5 %
t-SNE 43.5 %
```

```
autoencoder 15.6 %

8 columnas escaladas 43.4 %
proporción en el conjunto entero 4.7 %
autoencoder con semilla 1 12.5 %
autoencoder con semilla 2 14.8 %
```

Las manchas amarillas de las gráficas son otra cosa que la diferencia media, que promedia sobre todo el conjunto. Ese amarillo sale de que 965 renglones —el 4.7 %— marcan exactamente 5.00001, el máximo de la escala, y como el precio viene recortado ahí, por dentro del grupo no se distingue nada. Cuento entonces qué parte de los 20 vecinos de un renglón del máximo está también en el máximo.

t-SNE rodea a uno de ellos con 43.5 % de renglones del máximo, casi el mismo 43.4 % de las ocho columnas escaladas; UMAP alcanza 42.5 %. PCA se queda en 23.7 % y el autoencoder en 15.6 %, con 12.5 % y 14.8 % en sus otras dos semillas. El par que la diferencia media no ordenaba queda ordenado aquí, y al revés de como se veía allá: entre esos dos planos casi iguales, el que mejor reconstruye las ocho columnas es el que peor conserva el grupo del máximo. En la rejilla completa la tendencia no es limpia —`tanh` con 30 épocas reconstruye peor que PCA— así que la dejo entre esos dos.

```python
def geo_vecino(Z):
    v = vecinos(Z)
    dlat = X[v, 6] - X[:, 6][:, None]
    dlon = X[v, 7] - X[:, 7][:, None]
    return np.sqrt(dlat ** 2 + dlon ** 2).mean()


for nombre, Z in proyecciones:
    print(nombre, round(geo_vecino(Z), 4))
print()
print("8 columnas escaladas", round(geo_vecino(Xs), 4))
print("autoencoder con semilla 2",
      round(geo_vecino(latentes[("relu", 100, 2)]), 4))
```

```
PCA 0.7169
UMAP 0.5901
t-SNE 0.4438
autoencoder 0.6096

8 columnas escaladas 0.3648
autoencoder con semilla 2 0.7036
```

Latitude y Longitude están entre las ocho columnas, así que los cuatro métodos también aplanan un mapa de California. La misma cuenta de los vecinos, cambiando el precio por la distancia en grados, lo ordena igual: t-SNE 0.4438, UMAP 0.5901, autoencoder 0.6096 —0.7036 con la otra semilla— y PCA 0.7169, contra 0.3648 de las ocho columnas.

10. Dónde quedó California en el mapa de UMAP

El canal vertical que parte el mapa de UMAP en dos es lo bastante limpio como para preguntarle qué separa.

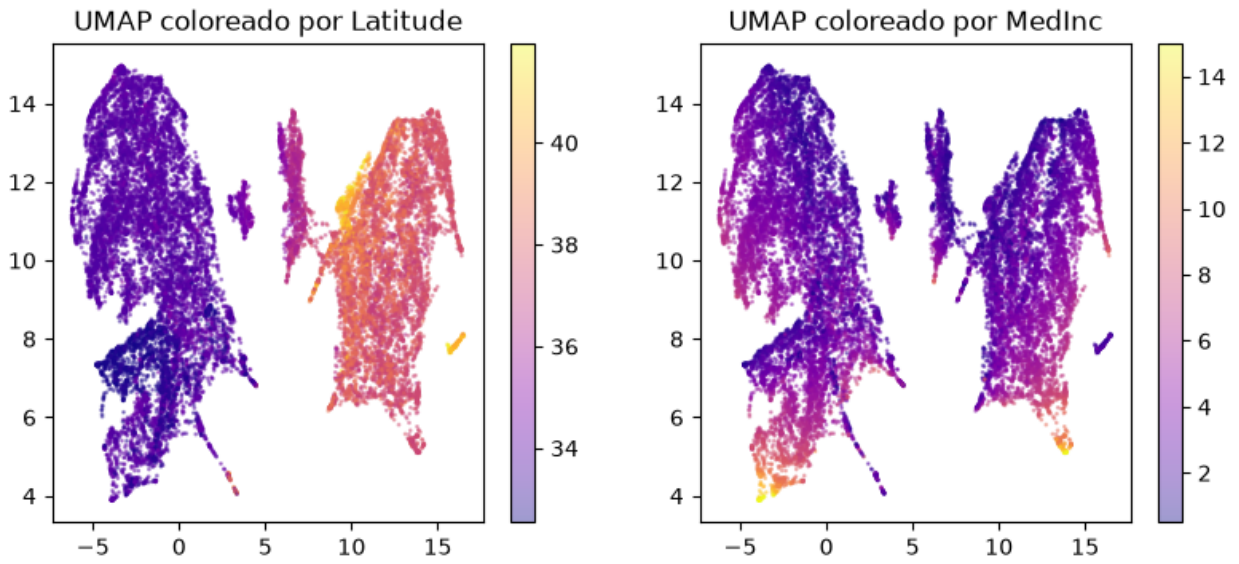
```
print("columna     horizontal vertical")
for j, c in [(6, "Latitude"), (7, "Longitude"), (0, "MedInc"),
             (1, "HouseAge")]:
    print(f"{c:11s}", round(np.corrcoef(Z_umap[:, 0], X[:, j])[0, 1], 3),
          round(np.corrcoef(Z_umap[:, 1], X[:, j])[0, 1], 3))
print("precio     ", round(np.corrcoef(Z_umap[:, 0], y)[0, 1], 3),
      round(np.corrcoef(Z_umap[:, 1], y)[0, 1], 3))

izq = Z_umap[:, 0] < 5
print()
print("bloque izquierdo:", izq.sum(), "renglones, latitud media",
      round(X[izq, 6].mean(), 2), "precio medio", round(y[izq].mean(), 3))
print("bloque derecho:", (~izq).sum(), "renglones, latitud media",
      round(X[~izq, 6].mean(), 2), "precio medio", round(y[~izq].mean(), 3))
print("de los renglones con latitud < 35, cae a la izquierda el",
      round(100 * izq[X[:, 6] < 35].mean(), 1), "%")
```

```
columna     horizontal vertical
Latitude    0.896 0.08
Longitude   -0.91 -0.082
MedInc      -0.047 -0.627
HouseAge    -0.002 0.613
precio      -0.072 -0.345

bloque izquierdo: 11597 renglones, latitud media 33.9 precio medio 2.198
bloque derecho: 9043 renglones, latitud media 37.86 precio medio 1.902
de los renglones con latitud < 35, cae a la izquierda el 100.0 %
```

```
fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.9))
for a_i, (col, nombre) in zip(ax, [(6, "Latitude"), (0, "MedInc")]):
    p = a_i.scatter(Z_umap[:, 0], Z_umap[:, 1], c=X[:, col], s=1,
                    alpha=0.4, cmap="plasma")
    a_i.set_title("UMAP coloreado por " + nombre)
    fig.colorbar(p, ax=a_i)
plt.show()
```



El eje horizontal correlaciona 0.896 con Latitude y −0.910 con Longitude: el sur contra el norte, ahora con el hueco dibujado. Cortando en 5, el bloque izquierdo son 11597 renglones de latitud media 33.90 y el derecho 9043 de 37.86, y el 100 % de los bloques al sur del paralelo 35 está a la izquierda. El eje vertical reparte entre ingreso y antigüedad, −0.627 con MedInc y 0.613 con HouseAge, y con el precio queda en −0.345, que es lo que lleva el amarillo al borde inferior de los dos bloques. Las dos gráficas de aquí arriba son el mismo mapa pintado con esas dos columnas. El precio medio de los dos bloques, 2.198 a la izquierda y 1.902 a la derecha, no encaja con la idea usual de que la bahía es lo caro; son datos de 1990.

11. Cierre

Para leer el precio en un plano, t-SNE (0.6612 de diferencia entre vecinos, 43.5 % de vecinos en el máximo alrededor de uno del máximo) y UMAP (0.7125 y 42.5 %) quedan por delante de PCA (0.9904 y 23.7 %) y del autoencoder (0.9720 y 15.6 %). Entre esos dos últimos la diferencia media no alcanza a ordenarlos, 0.018 contra una desviación de 0.016 entre semillas de la red, y solo la cuenta del máximo los separa, en contra del autoencoder.

Lo que separa a los dos primeros de los dos últimos es en qué se gastan los dos ejes. PCA y la red se reparten casi el mismo plano entre la geografía y el tamaño de la vivienda, que son las direcciones con más varianza y las que mejor reconstruyen las ocho columnas; a MedInc, la columna que va con el precio, le dejan 0.3 y 0.2 de correlación. UMAP dedica su eje vertical al ingreso y la antigüedad, y t-SNE no tiene ninguna dirección global que reportar: arma islas locales. Mi medida mira exactamente eso, vecindarios de veinte puntos, que es el terreno que esos dos métodos optimizan por construcción.

Escalar movió más que cualquier otra decisión, de 1.1281 a 0.6078 antes de proyectar, y el resto se decidió después, sobre columnas que ya pesaban lo mismo. Con Latitude y Longitude solas la vecindad da 0.4876 y ninguno de los cuatro mapas llega ahí, así que lo que aquí queda ordenado no es qué método aplana mejor, sino cuánto del vínculo entre las ocho columnas y el precio llega vivo a dos dimensiones. Con el precio recortado en 5.00001 para 965 renglones, la parte alta de la escala es la que peor puedo juzgar.